

武汉大学

《云容器实验》实验报告

姓 名：刘鼎琨
学 号：2023302121009
专 业：计算机科学与技术
学 院：计算机学院
指导教师：林馥老师

二〇二五年 九月

原创性声明

本人郑重声明：所呈交的论文（设计），是本人在指导教师的指导下，严格按照学校和学院有关规定完成的。除文中已经标明引用的内容外，本论文（设计）不包含任何其他个人或集体已发表及撰写的研究成果。对本论文（设计）做出贡献的个人和集体，均已在文中以明确方式标明。本人承诺在论文（设计）工作过程中没有伪造数据等行为。若在本论文（设计）中有侵犯任何方面知识产权的行为，由本人承担相应的法律责任。

作者签名：刘鼎琨

日期： 2025 年 9 月 18 日

摘要

本实验通过华为云平台系统开展了云容器技术综合实践，完整掌握了 Docker 容器技术的核心操作与容器化部署全流程。实验重点完成了 OA 系统的容器化改造，通过 Docker 环境配置、镜像构建与管理、SWR 仓库操作以及 CCE 集群部署等关键环节，成功实现了传统应用向云原生架构的转型。通过本次实践，深入理解了容器化技术在应用部署、资源管理和服务运维方面的优势，建立了完整的容器化应用部署知识体系，为后续云计算和云原生应用开发奠定了坚实基础。

关键词：云容器技术；Docker；镜像构建；SWR 仓库；CCE 集群；容器化部署

ABSTRACT

This experiment conducted comprehensive practice of cloud container technology through the Huawei Cloud platform, fully mastering the core operations of Docker container technology and the complete containerized deployment process. The experiment focused on the containerization transformation of the OA system, successfully achieving the transition from traditional applications to cloud-native architecture through key steps including Docker environment configuration, image construction and management, SWR repository operations, and CCE cluster deployment. Through this practice, we gained deep understanding of the advantages of containerization technology in application deployment, resource management, and service operation, establishing a complete knowledge system for containerized application deployment, which lays a solid foundation for subsequent cloud computing and cloud-native application development.

Keywords: Cloud Container Technology; Docker; Image Building; SWR Repository; CCE Cluster; Containerized Deployment

目 录

原创性声明.....	II
摘 要.....	1
ABSTRACT.....	2
1 实验概述.....	1
1.1 实验目的.....	1
1.2 实验总体思路.....	1
1.3 实验注意事项说明.....	2
2 实验环境准备.....	4
2.1 云资源申请与配置.....	4
2.2 Docker 环境准备.....	5
3 Docker 操作与应用实践.....	7
3.1 镜像管理操作.....	7
3.2 容器生命周期管理.....	8
3.3 使用参数配置非 Deamon 模式.....	10
3.4 使用 Dockerfile 配置非 Deamon 模式.....	12
4 OA 系统容器化部署.....	15
4.1 应用镜像构建.....	15
4.2 镜像仓库管理.....	17
4.3 CCE 集群部署.....	19
5 实验感受与总结.....	24
5.1 实验手册注意事项.....	24
5.2 实验感受.....	24
5.3 实验总结.....	24

1 实验概述

1.1 实验目的

本实验基于华为云平台开展云容器技术实践，旨在系统掌握 Docker 容器技术的核心操作与应用部署方法。通过完成 OA 系统的容器化改造与部署全过程，重点学习 Docker 镜像管理、容器操作、Dockerfile 编写等关键技术，并实践华为云容器服务 SWR 和 CCE 的实际应用。

实验将帮助我建立完整的容器化应用部署知识体系，理解容器技术在提升应用可移植性和简化部署流程方面的优势。通过从环境搭建到应用部署的全流程实践，培养容器化应用的实施能力，为后续深入学习和应用云原生技术奠定基础。

1.2 实验总体思路

本实验分为 Docker 技术学习和 OA 系统容器化部署两个核心部分，通过循序渐进的方式系统掌握云容器技术的完整应用流程。

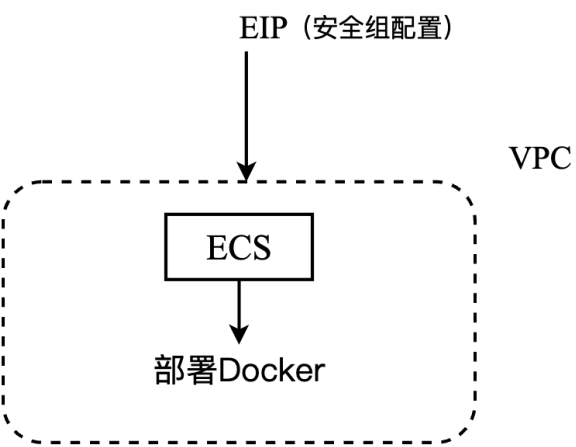


图 1.1 Docker 配置图

实验第一部分专注于 Docker 技术的学习与实践。如图 1.1 所示，实验配置只需基于华为云 ECS 实例构建 Docker 运行环境。首先完成基础配置工作，包括 docker-ce 安装、镜像源设置和环境验证。随后系统学习 Docker 核心操作技术，涵盖镜像管理、容器生命周期管理等基本语句使用。重点实践 Dockerfile 的编写与应用，通过构建 Nginx 镜像案例，掌握两种非 daemon 模式的配置方式：第一种采用运行参数直接配置，实现 80 端口的映射访问；第二种通过 Dockerfile 指

令配置，实现 81 端口的映射访问，从而深入理解容器化配置的多种实现方法。

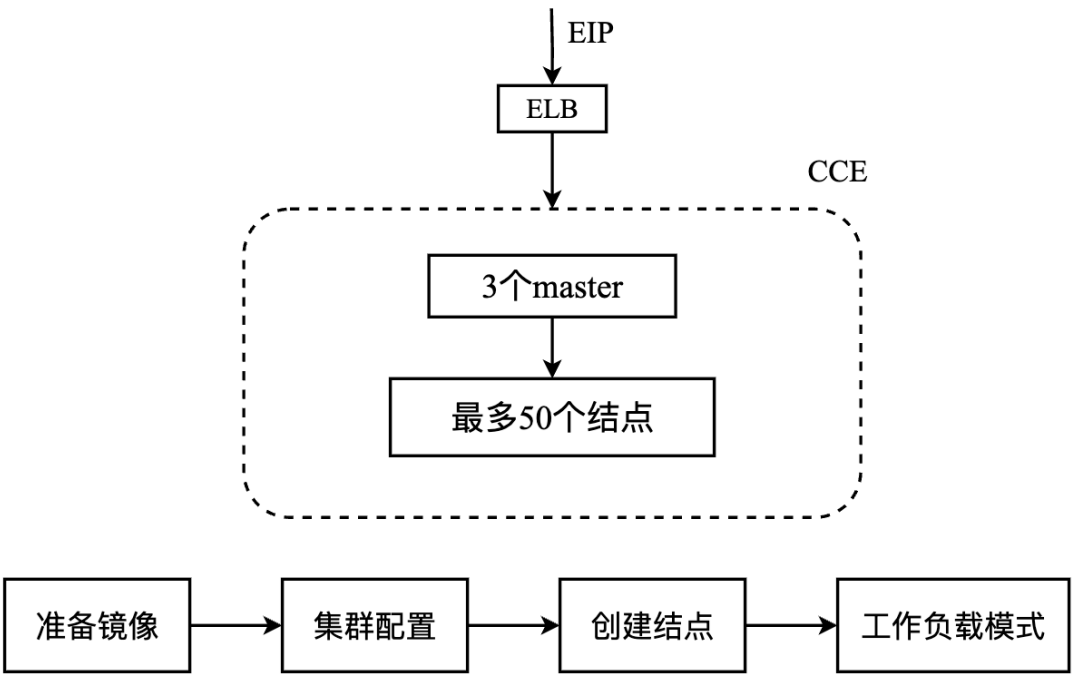


图 1.2 OA 系统容器化部署示意图

如图 1.2 所示，实验第二部分专注于 OA 系统的容器化部署实施，整体遵循镜像构建、仓库管理和集群部署三个阶段推进。首先进行应用镜像构建，基于前期 Docker 技术基础，完成 OA 系统的容器化改造。重点包括 Java 运行环境适配，使用 openjdk:8-alpine 基础镜像确保架构兼容，通过 Dockerfile 定义应用依赖和环境配置，构建优化的 OA 系统镜像。完成镜像构建后进入仓库管理阶段，将本地构建的镜像上传至华为云 SWR 服务，实现镜像的集中存储和版本管理。配置访问权限和同步策略，为集群部署提供可靠的镜像来源。最后进行 CCE 集群部署，在云容器引擎中创建托管集群，配置节点和网络环境。通过工作负载管理部署镜像，设置资源配额和副本数量，配置负载均衡实现流量分发，确保应用的高可用性和可扩展性。

1.3 实验注意事项说明

选取的 ECS 最好是 x86 架构，如果是鲲鹏的 arm 架构在构建 docker 镜像时需要转化。

Docker 环境的安装配置中，不能直接使用 `yum install -y docker` 命令进行安

装，此方式获取的旧版本 **Docker** 无法支持镜像加速等重要功能。应按照官方文档，通过安装 **docker-ce** 版本来获取稳定且功能完整的新版本 **Docker** 引擎，以确保实验环境的可靠性。

2 实验环境准备

2.1 云资源申请与配置

实验环境的云资源准备是整个项目的基础环节，需要按照标准流程完成各项服务的申请与配置。首先登录华为云控制台，进入弹性云服务器 ECS 管理页面开始资源创建。关键配置参数需要特别注意，如图 2.1 所示，计费模式选择按需计费，区域根据实验要求选择相应的可用区，CPU 架构必须选择 x86 计算以确保后续软件兼容性，规格选用通用计算增强型 c7.large.2 配置 2vCPUs 和 4GiB 内存，镜像选择 Ubuntu 22.04 64bit 版本以满足实验要求。

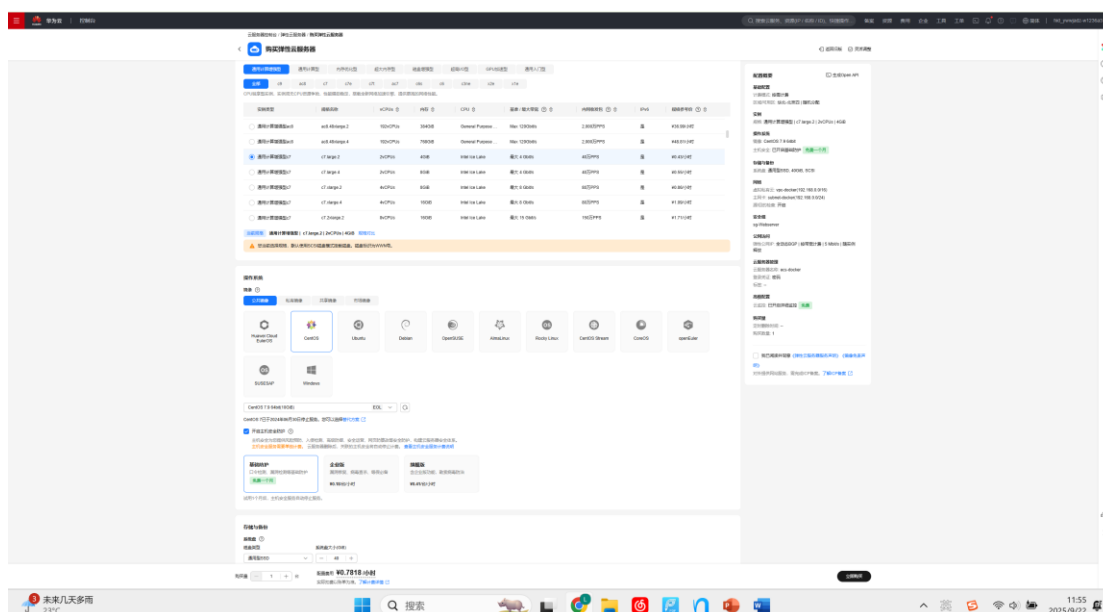


图 2.1 ECS 配置图

系统盘配置采用通用型 SSD，容量设置为 40GB 以满足基本存储需求。网络配置环节需要创建新的虚拟私有云 VPC，设置合适的网段地址，选择与实验需求匹配的子网。安全组规则需要精心配置，创建新的安全组并开放必要的端口，包括 22 端口用于 SSH 远程连接，80 和 8088 端口用于应用服务访问，同时配置 ICMP 协议允许 ping 测试。弹性公网 IP 选择按带宽计费模式，带宽大小设置为 5Mbit/s 以满足实验访问需求。

完成 ECS 实例创建后，还需要配置相应的网络策略和安全规则。在 VPC 控制台中确认网络配置正确，子网划分合理，路由表配置无误。安全组规则需要细化入方向和出方向策略，确保实验所需端口的通畅访问，同时保持网络安全的最小权限原则。所有配置完成后，记录实例的内网 IP 地址、弹性公网 IP 地址以及

登录凭证信息，为后续实验步骤做好准备。最终生成 ECS 实例如图 2.2 所示。



图 2.2 ECS 实例图

2.2 Docker 环境准备

Docker 环境的准备需要按照标准流程进行，首先通过 SSH 客户端连接到已创建的 ECS 实例。连接成功后更新系统软件包列表，确保系统处于最新状态。由于实验要求使用特定版本的 Docker，需要先卸载系统可能自带的旧版本 Docker 组件，包括 docker、docker-engine 等旧版本软件包。

安装必要的依赖工具包，包括 yum-utils、device-mapper-persistent-data 和 lvm2 等 Docker 所需的底层支持组件。添加 Docker 官方 yum 仓库源，由于网络访问考虑，建议使用国内镜像源进行替代以提高下载速度。具体操作是下载官方 repo 文件后，使用 sed 命令将仓库地址替换为国内镜像源地址。

完成仓库配置后更新 yum 包索引并安装 docker-ce 版本。安装过程中需要确认版本兼容性，选择与 Ubuntu 22.04 系统匹配的 docker-ce 版本。安装完成后启动 Docker 服务并设置为开机自启，通过 systemctl 命令启用并启动 docker 服务。

如图 2.3 所示，为了验证 Docker 安装是否成功，运行测试用例 hello-world 镜像，确认能够正常拉取镜像并运行容器。配置镜像加速器以提高后续镜像下载速度，创建或修改/etc/docker/daemon.json 文件，添加国内镜像加速器地址。重新加载服务配置并重启 Docker 使配置生效，最后通过 docker info 命令检查配置是否正确应用。

```
[root@ecs-docker ~]# docker run swr.cn-north-4.myhuaweicloud.com/library/hello-world
Unable to find image 'swr.cn-north-4.myhuaweicloud.com/library/hello-world:latest' locally
latest: Pulling from library/hello-world
d1725b59e92d: Pull complete
Digest: sha256:1a6fd470b9ce10849be79e99529a88371dff60c60aab424c077007f6979b4812
Status: Downloaded newer image for swr.cn-north-4.myhuaweicloud.com/library/hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

[root@ecs-docker ~]#
```

图 2.3 hello-world 镜像运行图

3 Docker 操作与应用实践

3.1 镜像管理操作

镜像管理是 Docker 技术的基础操作环节，如图 3.1 所示，我通过实验学习了镜像的获取、查询、检查和删除等核心操作。下面时具体的操作步骤，首先需要进行镜像的获取，通过 `docker pull` 命令从镜像仓库下载所需镜像。由于直接从国外 Docker Hub 下载可能存在网络限制，建议配置华为 SWR 镜像加速器以提高下载速度。具体操作是编辑 `/etc/docker/daemon.json` 文件，添加镜像加速器地址后重启 `docker` 服务。

获取镜像后需要进行查询操作，使用 `docker images` 或 `docker image ls` 命令可以查看本地已下载的镜像列表。该命令会显示镜像的仓库名、标签、镜像 ID、创建时间及所占用的空间等详细信息。通过添加过滤参数可以查询特定镜像，例如 `docker images nginx` 可以专门查看 `nginx` 相关的镜像。

镜像的详细信息的获取可以通过 `docker inspect` 命令实现，该命令能够查看镜像的完整元数据，包括支持的系统架构、环境变量配置、层信息等。这对于了解镜像的详细特性和排查问题非常有帮助。同时使用 `docker system df` 命令可以查看镜像、容器和卷的磁盘使用情况，方便进行存储空间管理。

当需要删除镜像时，使用 `docker rmi` 或 `docker image rm` 命令，可以通过镜像 ID 或仓库名加标签的方式指定要删除的镜像。如果镜像已经被容器使用，需要先删除相关容器或添加 `-f` 参数强制删除。在删除镜像前建议确认镜像是否确实不再需要，避免误删带来的重新下载时间消耗。

```

[root@ecs-docker ~]# docker rename 4853eb2deb9e myhttpd
[root@ecs-docker ~]# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
4853eb2deb9e   httpd     "httpd-foreground"      4 minutes ago Up 36 seconds 80/tcp       myhttpd
[root@ecs-docker ~]# docker run ubuntu:14.04 /bin/echo 'Hello world'
Unable to find image 'ubuntu:14.04' locally
14.04: Pulling from library/ubuntu
2e6e20c8e2e6: Pull complete
0551a797c01d: Pull complete
512123a064da: Pull complete
Digest: sha256:64483f3496c1373bfd55348e88694d1c4d0c9b660dee6bfef5e12f43b9933b30
Status: Downloaded newer image for ubuntu:14.04
Hello world
[root@ecs-docker ~]# docker run -it ubuntu:14.04 /bin/bash
root@48e118792028:/# pwd
/
root@48e118792028:/# ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
root@48e118792028:/# exit
exit
[root@ecs-docker ~]# docker run ubuntu:14.04 /bin/sh -c "while true; do echo hello world; sleep 1; done"
hello world
hello world
hello world
^C[root@ecs-docker ~]# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
4853eb2deb9e   httpd     "httpd-foreground"      7 minutes ago Up 3 minutes 80/tcp       myhttpd
[root@ecs-docker ~]# docker logs 485
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directive globally to suppress
this message
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directive globally to suppress
this message
[Mon Sep 22 05:12:01.641022 2025] [mpm_event:notice] [pid 1:tid 1] AH00489: Apache/2.4.65 (Unix) configured -- resuming normal operations
[Mon Sep 22 05:12:01.641133 2025] [core:notice] [pid 1:tid 1] AH00094: Command line: 'httpd -D FOREGROUND'
[Mon Sep 22 05:12:41.916973 2025] [mpm_event:notice] [pid 1:tid 1] AH00492: caught SIGWINCH, shutting down gracefully
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directive globally to suppress
this message
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directive globally to suppress
this message
[Mon Sep 22 05:13:06.029847 2025] [mpm_event:notice] [pid 1:tid 1] AH00489: Apache/2.4.65 (Unix) configured -- resuming normal operations
[Mon Sep 22 05:13:06.029969 2025] [core:notice] [pid 1:tid 1] AH00094: Command line: 'httpd -D FOREGROUND'
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directive globally to suppress
this message
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directive globally to suppress
this message
[Mon Sep 22 05:14:35.287813 2025] [mpm_event:notice] [pid 1:tid 1] AH00489: Apache/2.4.65 (Unix) configured -- resuming normal operations
[Mon Sep 22 05:14:35.287910 2025] [core:notice] [pid 1:tid 1] AH00094: Command line: 'httpd -D FOREGROUND'
[root@ecs-docker ~]# docker logs fd9662ff976a
Error response from daemon: No such container: fd9662ff976a
[root@ecs-docker ~]# docker logs fd9662ff976a^C
[root@ecs-docker ~]# docker run -dit ubuntu:14.04
f56648f531ee9d18bb23597c72f783fa1b79f1dc82dfacc6c1863fdbbeefc5953
[root@ecs-docker ~]# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
f56648f531ee   ubuntu:14.04   "/bin/bash"             6 seconds ago Up 5 seconds 80/tcp       stoic_agnes1
4853eb2deb9e   httpd     "httpd-foreground"      7 minutes ago Up 4 minutes 80/tcp       myhttpd
[root@ecs-docker ~]# docker attach f
^C
root@f56648f531ee:/# docker attach f56648f531ee
bash: docker: command not found
root@f56648f531ee:/# ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
root@f56648f531ee:/# exit
exit
[root@ecs-docker ~]# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
4853eb2deb9e   httpd     "httpd-foreground"      9 minutes ago Up 5 minutes 80/tcp       myhttpd
[root@ecs-docker ~]#

```

图 3.1 docker 镜像管理操作实操图

3.2 容器生命周期管理

容器生命周期管理涵盖了从创建到销毁的完整过程，是 Docker 技术的核心应用环节。如图 3.2 所示，我充分学习并掌握了这些操作。

容器创建使用 docker create 命令，该命令基于指定镜像创建一个新的容器但

并不立即启动。若本地不存在指定镜像，Docker 会自动从镜像仓库拉取最新版本的镜像。创建完成后可以通过 `docker ps -a` 命令查看所有容器状态，包括已创建但未运行的容器。

容器启动使用 `docker start` 命令，通过指定容器 ID 或名称来启动已创建的容器。启动后容器进入运行状态，可以通过 `docker ps` 命令查看运行中的容器信息。对于需要直接创建并运行容器的情况，使用 `docker run` 命令更加便捷，该命令会完成从镜像拉取到容器启动的全过程。

容器运行期间可能需要进行状态调整，`docker pause` 命令可以暂停运行中的容器，暂停后容器内的进程会被临时冻结。使用 `docker unpause` 命令可以恢复暂停的容器继续运行。当需要强制停止容器时，使用 `docker kill` 命令会立即终止容器进程，而 `docker stop` 命令则会先发送停止信号给容器内的主进程，等待其正常退出。

对于需要与容器进行交互的场景，使用 `docker exec` 命令可以在运行中的容器内执行命令，配合 `-it` 参数可以开启交互式终端。这在需要调试或查看容器内部状态时非常有用。`docker attach` 命令则可以连接到正在运行的容器的主进程，观察其输出或进行交互。

容器停止后的清理工作同样重要，使用 `docker rm` 命令可以删除已停止的容器，添加 `-f` 参数可以强制删除运行中的容器。定期清理不再使用的容器可以释放磁盘空间，保持系统整洁。通过 `docker container prune` 命令可以一次性删除所有已停止的容器，提高清理效率。

```
[root@ecs-docker ~]# docker run -d -p 5000:5000 --restart=always --name registry registry
Unable to find image 'registry:latest' locally
latest: Pulling from library/registry
0368fd46e3c6: Pull complete
56cbcc5405f0: Pull complete
b65b097f3c34: Pull complete
60999dff384a: Pull complete
e6a3228d448c: Pull complete
Digest: sha256:3725021071ec9383eb3d87ddbdf9ed602439b3f7c958c9c2fb941049ea6531d
Status: Downloaded newer image for registry:latest
524973f61b2702ef90953b9c0526f0ad662a3f3e2f650174c2c5b14ae5b262ef
[root@ecs-docker ~]# docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
524973f61b27   registry   "/entrypoint.sh /etc..." 10 seconds ago Up 10 seconds 0.0.0.0:5000->5000/tcp, :::5000->5000/tcp registry
4853eb2deb9e   httpd      "httpd-foreground"        10 minutes ago Up 7 minutes   80/tcp                               myhttpd
[root@ecs-docker ~]# docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
nginx                latest      41f689c20910  5 weeks ago   192MB
httpd                latest      65005131d37e  6 weeks ago   117MB
registry             latest      3c52eedeec80  5 months ago  57.7MB
ubuntu               14.04      13b66b487594  4 years ago   197MB
swr.cn-north-4.myhuaweicloud.com/library/hello-world latest 4ab4c602aa5e  7 years ago   1.84kB
[root@ecs-docker ~]# docker tag ubuntu:14.04 127.0.0.1:5000/myubuntu:14.04
[root@ecs-docker ~]# docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
nginx                latest      41f689c20910  5 weeks ago   192MB
httpd                latest      65005131d37e  6 weeks ago   117MB
registry             latest      3c52eedeec80  5 months ago  57.7MB
127.0.0.1:5000/myubuntu 14.04      13b66b487594  4 years ago   197MB
ubuntu               14.04      13b66b487594  4 years ago   197MB
swr.cn-north-4.myhuaweicloud.com/library/hello-world latest 4ab4c602aa5e  7 years ago   1.84kB
[root@ecs-docker ~]# docker push 127.0.0.1:5000/myubuntu:14.04
The push refers to repository [127.0.0.1:5000/myubuntu]
83109fa660b2: Pushed
30d3c4334a23: Pushed
f2fa9f4cf8fd: Pushed
14.04: digest: sha256:881afbae521c910f764f7187dbfbca3cc10c26f8bafa458c76dda009a901c29d size: 945
[root@ecs-docker ~]# curl 127.0.0.1:5000/v2/_catalog
{"repositories":["myubuntu"]}
[root@ecs-docker ~]# ^C
[root@ecs-docker ~]# docker image rm 127.0.0.1:5000/myubuntu:14.04^C
[root@ecs-docker ~]# docker image rm 127.0.0.1:5000/myubuntu:14.04
Untagged: 127.0.0.1:5000/myubuntu:14.04
Untagged: 127.0.0.1:5000/myubuntu@sha256:881afbae521c910f764f7187dbfbca3cc10c26f8bafa458c76dda009a901c29d
[root@ecs-docker ~]# docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
nginx                latest      41f689c20910  5 weeks ago   192MB
httpd                latest      65005131d37e  6 weeks ago   117MB
registry             latest      3c52eedeec80  5 months ago  57.7MB
ubuntu               14.04      13b66b487594  4 years ago   197MB
swr.cn-north-4.myhuaweicloud.com/library/hello-world latest 4ab4c602aa5e  7 years ago   1.84kB
[root@ecs-docker ~]# docker pull 127.0.0.1:5000/myubuntu:14.04
14.04: Pulling from myubuntu
Digest: sha256:881afbae521c910f764f7187dbfbca3cc10c26f8bafa458c76dda009a901c29d
Status: Downloaded newer image for 127.0.0.1:5000/myubuntu:14.04
127.0.0.1:5000/myubuntu:14.04
[root@ecs-docker ~]# docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
nginx                latest      41f689c20910  5 weeks ago   192MB
httpd                latest      65005131d37e  6 weeks ago   117MB
registry             latest      3c52eedeec80  5 months ago  57.7MB
127.0.0.1:5000/myubuntu 14.04      13b66b487594  4 years ago   197MB
ubuntu               14.04      13b66b487594  4 years ago   197MB
swr.cn-north-4.myhuaweicloud.com/library/hello-world latest 4ab4c602aa5e  7 years ago   1.84kB
[root@ecs-docker ~]#
```

图 3.2 容器生命周期管理实操图

3.3 使用参数配置非 Deamon 模式

在 Docker 容器部署实践中,配置非 Daemon 模式是确保服务持续运行的关键

技术环节。通过运行参数配置非 Daemon 模式主要是在容器启动时，通过 docker run 命令附加特定的参数来实现服务的持久化运行。

以 Nginx 服务器为例，常规启动方式会导致容器立即退出，因为 Nginx 默认以 Daemon 模式运行。为解决这个问题，需要在启动容器时使用特定的运行参数。具体操作方法是执行 docker run 命令时，在镜像名称后添加 nginx -g "daemon off;" 参数。这个参数的作用是告诉 Nginx 以前台模式运行，而不是作为守护进程在后台运行。

如图 3.3 所示，实际操作中，首先需要拉取 Nginx 官方镜像，使用 docker pull nginx 命令获取最新的 Nginx 镜像。然后运行容器并配置端口映射，例如将容器的 80 端口映射到主机的 80 端口，完整命令为：docker run -d -p 80:80 nginx nginx -g "daemon off;"。这里的 -d 参数表示以后台模式运行容器，而 -p 参数则完成端口映射配置。

```
root@ecs-docker ~]# docker pull nginx:alpine-perl
alpine-perl: Pulling from library/nginx
9824c27679d3: Pull complete
8bc572a340ec: Pull complete
403e3f251637: Pull complete
9adfbac99cb7: Pull complete
7a8a46741e18: Pull complete
c9ebe2ff2d2c: Pull complete
a992fbc61ecc: Pull complete
b1ff4086f02: Pull complete
66859d91676f: Pull complete
Digest: sha256:b04b512a3daf265299d237dcef02909b4f3fa998800830d1beef22e29d36ee97
Status: Downloaded newer image for nginx:alpine-perl
docker.io/library/nginx:alpine-perl
root@ecs-docker ~]# pwd
/root
root@ecs-docker ~]# mkdir nginx_demo
root@ecs-docker ~]# ls
nginx_demo
root@ecs-docker ~]# vim Dockerfile
root@ecs-docker ~]# docker build -t my_nginx:v1 .
[+] Building 0.5s (6/6) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 160B
=> [internal] load metadata for docker.io/library/nginx:alpine-perl
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/2] FROM docker.io/library/nginx:alpine-perl
=> [2/2] RUN chmod -R 755 /usr/lib/nginx/
=> exporting to image
=> => exporting layers
=> => writing image sha256:573d7b689a5d9ebcb4887f054fc305c4610753cd546ebc258d8eb9fad4c1028
=> => naming to docker.io/library/my_nginx:v1
root@ecs-docker ~]# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
my_nginx             v1                 573d7b689a5d       34 seconds ago     98.6MB
nginx                latest             41f689c20910       5 weeks ago        192MB
nginx                alpine-perl       a6dd63693915       5 weeks ago        91MB
httpd                latest             65005131d37e       6 weeks ago        117MB
registry             latest             3c52eede8c80       5 months ago       57.7MB
127.0.0.1:5000/myubuntu 14.04             13b66b487594       4 years ago        197MB
ubuntu               14.04             13b66b487594       4 years ago        197MB
swr.cn-north-4-myhuaweicloud.com/library/hello-world latest          4ab4c602aa5e       7 years ago        1.84kB
root@ecs-docker ~]# docker run -d -p 80:80 my_nginx:v1 /usr/sbin/nginx -g "daemon off;"
47aa3582c15ace0095dcd624b4d80fc5efa73e87ca9601860dc0dcabc07f1a
root@ecs-docker ~]# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
47aa3582c15   my_nginx:v1 "/docker-entrypoint..." 14 seconds ago Up 13 seconds 0.0.0.0:80->80/tcp, :::80->80/tcp   sharp_poincare
524973f61b27 registry   "/entrypoint.sh /etc..." 14 minutes ago Up 14 minutes 0.0.0.0:5000->5000/tcp, :::5000->5000/tcp registry
4852eb2deb9e httpd      "httpd-foreground"       25 minutes ago Up 21 minutes 80/tcp                             myhttpd
root@ecs-docker ~]#
```

图 3.3 使用参数配置非 daemon 模式实操图

验证配置是否成功的方法是通过 `docker ps` 命令查看容器运行状态，确认容器处于 Up 状态且运行时间持续增长。同时可以通过浏览器访问宿主机的 IP 地址，检查 Nginx 欢迎页面是否能正常显示（图 3.4）。这种配置方式的优势在于简单直接，不需要修改镜像本身的配置，适用于临时性的测试环境或快速部署场景。

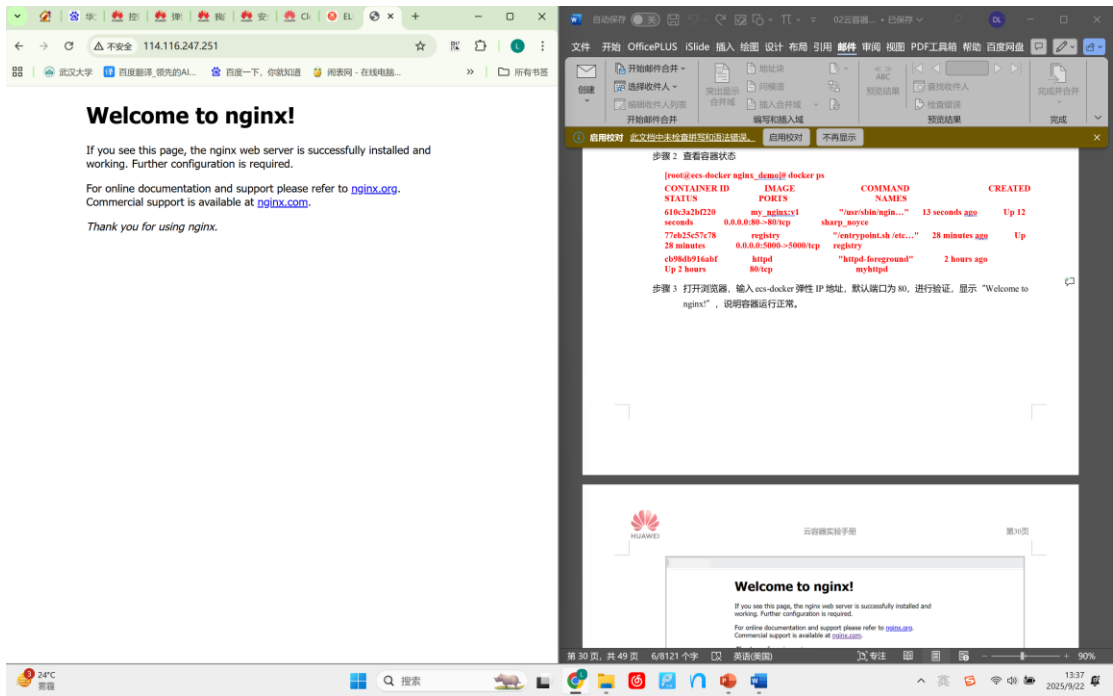


图 3.4 Nginx 欢迎页面验证图

3.4 使用 Dockerfile 配置非 Daemon 模式

使用 Dockerfile 配置非 Daemon 模式是一种更规范、可重复的部署方式，通过将运行参数固化到镜像构建过程中，实现部署的标准化和自动化。这种方法适用于生产环境或需要频繁部署的场景。

如图 3.5 所示，在 Dockerfile 中配置非 Daemon 模式主要通过 CMD 或 ENTRYPOINT 指令实现。以 Nginx 为例，创建 Dockerfile 文件并添加以下内容：首先使用 FROM nginx 指定基础镜像，然后通过 CMD ["nginx", "-g", "daemon off;"] 设置容器启动时自动执行的命令。这种配置方式确保每次从该镜像启动容器时都

会自动以非 Daemon 模式运行。

```
root@114.116.247.251 x
Last failed login: Mon Sep 22 14:00:37 CST 2025 from 14.103.118.113 on ssh:notty
There were 5 failed login attempts since the last successful login.
Last login: Mon Sep 22 13:02:30 2025 from 121.36.59.153

Welcome to Huawei Cloud Service

[root@ecs-docker ~]# ls
Dockerfile  nginx_demo
[root@ecs-docker ~]# vim Dockerfile
[root@ecs-docker ~]# docker build -t my_nginx:v2 .
[+] Building 0.1s (6/6) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 208B
=> [internal] load metadata for docker.io/library/nginx:alpine-perl
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/2] FROM docker.io/library/nginx:alpine-perl
=> CACHED [2/2] RUN chmod -R 755 /usr/lib/nginx/
=> exporting to image
=> => exporting layers
=> => writing image sha256:a0f561e70d2fcd00e415f0de64b1e23f4f3c4a6026d66e9b26ae356743d36e2
=> => naming to docker.io/library/my_nginx:v2
[root@ecs-docker ~]# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
my_nginx             v2                 a0f561e70d2f      34 minutes ago     98.6MB
my_nginx             v1                 573d7b689a5d      34 minutes ago     98.6MB
nginx                latest             41f689c20910      5 weeks ago        192MB
nginx                alpine-perl       a6dd63693915      5 weeks ago        91MB
httpd                latest             65005131d37e      6 weeks ago        117MB
registry             latest             3c52eedec80       5 months ago       57.7MB
ubuntu               14.04             13b66b487594      4 years ago        197MB
127.0.0.1:5000/myubuntu 14.04             13b66b487594      4 years ago        197MB
swr.cn-north-4.myhuaweicloud.com/library/hello-world latest             4ab4c602aa5e      7 years ago        1.84kB
[root@ecs-docker ~]# docker run -d -p 81:80 my_nginx:v2
49446df1280725f0e0c7821ff833835ae1872209bf673e94c69b3176f89b1dab
[root@ecs-docker ~]# docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
49446df12807   my_nginx:v2 "/docker-entrypoint..." 7 seconds ago Up 5 seconds 0.0.0.0:81->80/tcp, :::81->80/tcp   relaxed_brattain
f47aa3582c15   my_nginx:v1 "/docker-entrypoint..." 33 minutes ago Up 33 minutes 0.0.0.0:80->80/tcp, :::80->80/tcp   sharp_poincare
524973f61b27   registry  "/entrypoint.sh /etc..." 47 minutes ago Up 47 minutes 0.0.0.0:5000->5000/tcp, :::5000->5000/tcp registry
4853eb2deb9e   httpd     "httpd-foreground"       58 minutes ago Up 54 minutes 80/tcp                             myhttpd
[root@ecs-docker ~]#
```

图 3.5 使用 Dockerfile 配置非 daemon 模式实操图

构建镜像时使用 `docker build -t my-nginx:v2 .` 命令，其中 `-t` 参数指定镜像名称和标签。构建完成后，运行容器只需使用简单的 `docker run -d -p 81:80 my-nginx:v1` 命令，不再需要附加额外的运行参数。这种方式简化了部署命令，降低了出错概率，提高了部署效率。

验证 Dockerfile 配置效果时，除了检查容器运行状态外，还需要确认镜像的元数据信息。使用 `docker inspect` 命令可以查看镜像的配置详情，包括 `Cmd` 字段是否正确设置为非 Daemon 模式参数。与参数配置方式相比，Dockerfile 方式的优势在于配置的可维护性和可移植性更好，更适合团队协作和持续集成环境。

两种配置方式各有适用场景：参数配置方式灵活快捷，适合临时测试；Dockerfile 方式规范可靠，适合生产部署。在实际项目中可以根据具体需求选择合适的配置方式，或者将两种方式结合使用，既保证开发调试的便利性，又确保生产环境的稳定性。

最终使用 Dockerfile 配置非 daemon 模式的 81 端口访问结果如图 3.6 所示。

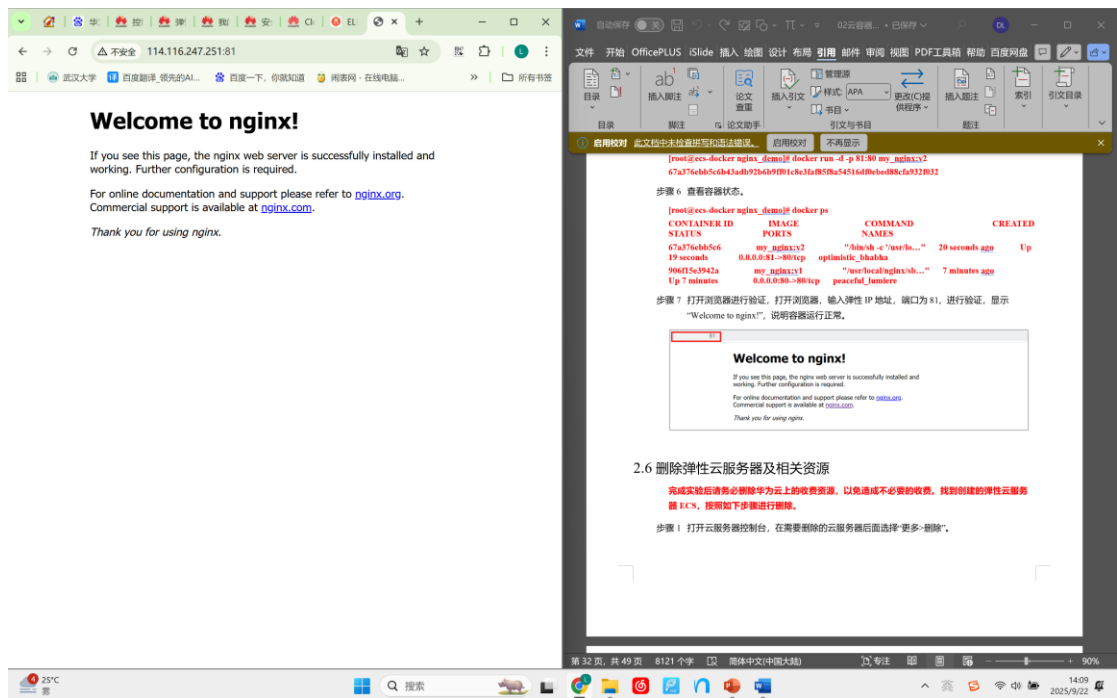


图 3.4 81 端口 Nginx 欢迎页面验证图

4 OA 系统容器化部署

4.1 应用镜像构建

OA 系统容器化部署的首要环节是应用镜像的构建，这一过程需要基于前期 Docker 技术基础，将传统的 OA 应用进行容器化改造。镜像构建的质量直接关系到后续部署和运行的稳定性，因此需要严格按照规范流程操作。

构建过程开始前需要完成基础环境准备。如图 4.1 所示，首先从代码仓库获取 OA 系统项目源代码，使用 `git clone` 命令下载到本地工作目录。随后进入项目目录进行依赖环境检查，确认 Java 版本和 Maven 配置符合项目要求。通过 `mvn clean package` 命令完成项目编译打包，生成可执行的 JAR 文件，这是后续构建容器镜像的基础材料。

```
[root@ecs-oa maven]# java -version
-bash: java: command not found
[root@ecs-oa maven]# sudo yum install -y java-1.8.0-openjdk-devel
Loaded plugins: fastestmirror
Determining fastest mirrors
base                                     | 3.6 kB  00:00:00
epel                                    | 4.3 kB  00:00:00
extras                                 | 2.9 kB  00:00:00
updates                                | 2.9 kB  00:00:00
(1/7): base/7/x86_64/group_gz          | 153 kB  00:00:00
(2/7): epel/x86_64/updateinfo          | 1.0 MB  00:00:00
(3/7): base/7/x86_64/primary_db        | 6.1 MB  00:00:00
(4/7): epel/x86_64/group               | 399 kB  00:00:00
(5/7): epel/x86_64/primary_db          | 8.7 MB  00:00:00
(6/7): extras/7/x86_64/primary_db      | 253 kB  00:00:00
(7/7): updates/7/x86_64/primary_db     | 27 MB  00:00:00
Resolving Dependencies
--> Running transaction check
--> Package java-1.8.0-openjdk-devel.x86_64 1:1.8.0.412.b08-1.el7_9 will be installed
--> Processing Dependency: java-1.8.0-openjdk(x86-64) = 1:1.8.0.412.b08-1.el7_9 for package: 1:java-1.8.0-openjdk-devel-1.8.0.412.b08-1.el7_9.x86_64
--> Processing Dependency: libjvm.so()(64bit) for package: 1:java-1.8.0-openjdk-devel-1.8.0.412.b08-1.el7_9.x86_64
--> Processing Dependency: libjava.so()(64bit) for package: 1:java-1.8.0-openjdk-devel-1.8.0.412.b08-1.el7_9.x86_64
--> Processing Dependency: libx11.so.6()(64bit) for package: 1:java-1.8.0-openjdk-devel-1.8.0.412.b08-1.el7_9.x86_64
--> Running transaction check
--> Package java-1.8.0-openjdk.x86_64 1:1.8.0.412.b08-1.el7_9 will be installed
--> Processing Dependency: xorg-x11-fonts-Type1 for package: 1:java-1.8.0-openjdk-1.8.0.412.b08-1.el7_9.x86_64
--> Processing Dependency: libjpeg.so.62(LIBJPEG_6.2)(64bit) for package: 1:java-1.8.0-openjdk-1.8.0.412.b08-1.el7_9.x86_64
--> Processing Dependency: libXcomposite(x86-64) for package: 1:java-1.8.0-openjdk-1.8.0.412.b08-1.el7_9.x86_64
--> Processing Dependency: gtk2(x86-64) for package: 1:java-1.8.0-openjdk-1.8.0.412.b08-1.el7_9.x86_64
--> Processing Dependency: fontconfig(x86-64) for package: 1:java-1.8.0-openjdk-1.8.0.412.b08-1.el7_9.x86_64
--> Processing Dependency: libjpeg.so.62()(64bit) for package: 1:java-1.8.0-openjdk-1.8.0.412.b08-1.el7_9.x86_64
--> Processing Dependency: libgif.so.4()(64bit) for package: 1:java-1.8.0-openjdk-1.8.0.412.b08-1.el7_9.x86_64
--> Processing Dependency: libXtst.so.6()(64bit) for package: 1:java-1.8.0-openjdk-1.8.0.412.b08-1.el7_9.x86_64
--> Processing Dependency: libXrender.so.1()(64bit) for package: 1:java-1.8.0-openjdk-1.8.0.412.b08-1.el7_9.x86_64
--> Processing Dependency: libXi.so.6()(64bit) for package: 1:java-1.8.0-openjdk-1.8.0.412.b08-1.el7_9.x86_64
--> Processing Dependency: libXext.so.6()(64bit) for package: 1:java-1.8.0-openjdk-1.8.0.412.b08-1.el7_9.x86_64
--> Package java-1.8.0-openjdk-headless.x86_64 1:1.8.0.412.b08-1.el7_9 will be installed
--> Processing Dependency: tzdata-java >= 2023d for package: 1:java-1.8.0-openjdk-headless-1.8.0.412.b08-1.el7_9.x86_64
--> Processing Dependency: copy-jdk-configs >= 3.3 for package: 1:java-1.8.0-openjdk-headless-1.8.0.412.b08-1.el7_9.x86_64
--> Processing Dependency: psc-lite-libs(x86-64) for package: 1:java-1.8.0-openjdk-headless-1.8.0.412.b08-1.el7_9.x86_64
```

图 4.1 应用镜像构建实操图

Dockerfile 的编写是镜像构建的核心环节。文件必须以 FROM 指令开头，指定基础镜像为 `openjdk:8-alpine`，确保使用 x86 架构且体积较小的基础环境。接下来通过 MAINTAINER 指令添加维护者信息，便于后续镜像管理。使用 ADD 指

令将编译好的 `oasys.jar` 文件添加到镜像中，放置在合适的工作目录下。

通过 `EXPOSE` 指令声明容器运行时需要暴露的端口号，OA 系统通常使用 8088 端口提供 web 服务。最后使用 `ENTRYPOINT` 指令定义容器启动时执行的命令，配置 `java -jar` 命令启动应用服务。整个 `Dockerfile` 需要保持简洁高效，减少不必要的层数，以减小最终镜像的体积。

镜像构建命令的执行需要特别注意版本管理。使用 `docker build` 命令构建镜像，通过 `-t` 参数指定镜像名称和标签，例如 `oa-system:1.0`。构建过程中 Docker 引擎会逐条执行 `Dockerfile` 中的指令，每成功执行一条指令就会生成一个中间镜像层。构建完成后使用 `docker images` 命令验证镜像是否成功创建，确认镜像大小、创建时间等信息符合预期。

如图 4.2 所示，为了确保镜像的可用性，需要进行简单的功能验证。通过 `docker run` 命令启动测试容器，检查应用是否正常启动并监听指定端口。可以使用 `curl` 命令或浏览器访问服务接口，验证应用功能是否正常。同时检查容器日志输出，确认没有错误信息，确保镜像构建过程完整正确。

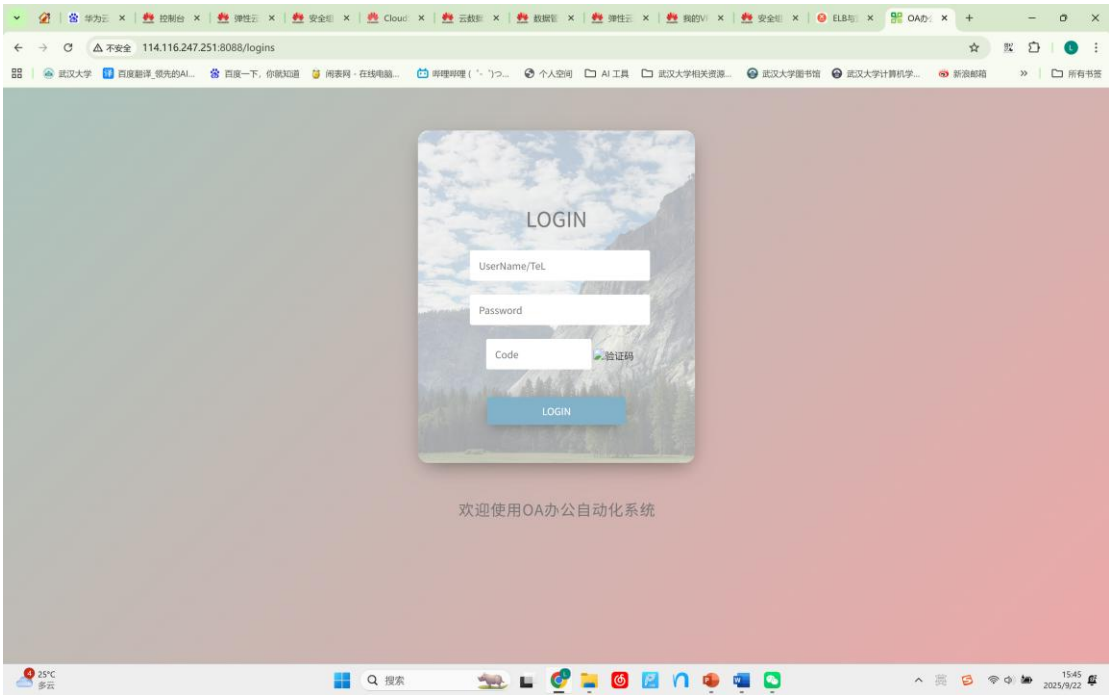


图 4.2 镜像功能验证图

镜像构建完成后，还需要进行必要的优化工作。包括清理构建过程中产生的临时文件，优化镜像层结构以减少体积，添加必要的元数据标签等。这些优化措施能够提高镜像的部署效率和运行性能，为后续的容器化部署奠定良好基础。

4.2 镜像仓库管理

镜像仓库管理是容器化部署流程中的关键环节，主要负责镜像的存储、版本管理和分发。本实验采用华为云 SWR（SoftWare Repository for Container）服务作为镜像仓库，为 OA 系统提供可靠的镜像托管和分发服务。

首先需要完成 SWR 服务的初始化配置。登录华为云控制台后进入容器镜像服务页面，创建独立的组织管理空间用于存放 OA 系统镜像。组织名称需要遵循命名规范，通常采用项目名称或团队标识作为组织名，便于后续的镜像分类和管理。组织创建完成后配置访问权限，根据团队协作需求设置不同的操作权限，确保镜像的安全性和可用性。

如图 4.3 所示，接着需要配置镜像加速器，这部分实测 docker-ce 版本的支持更好。

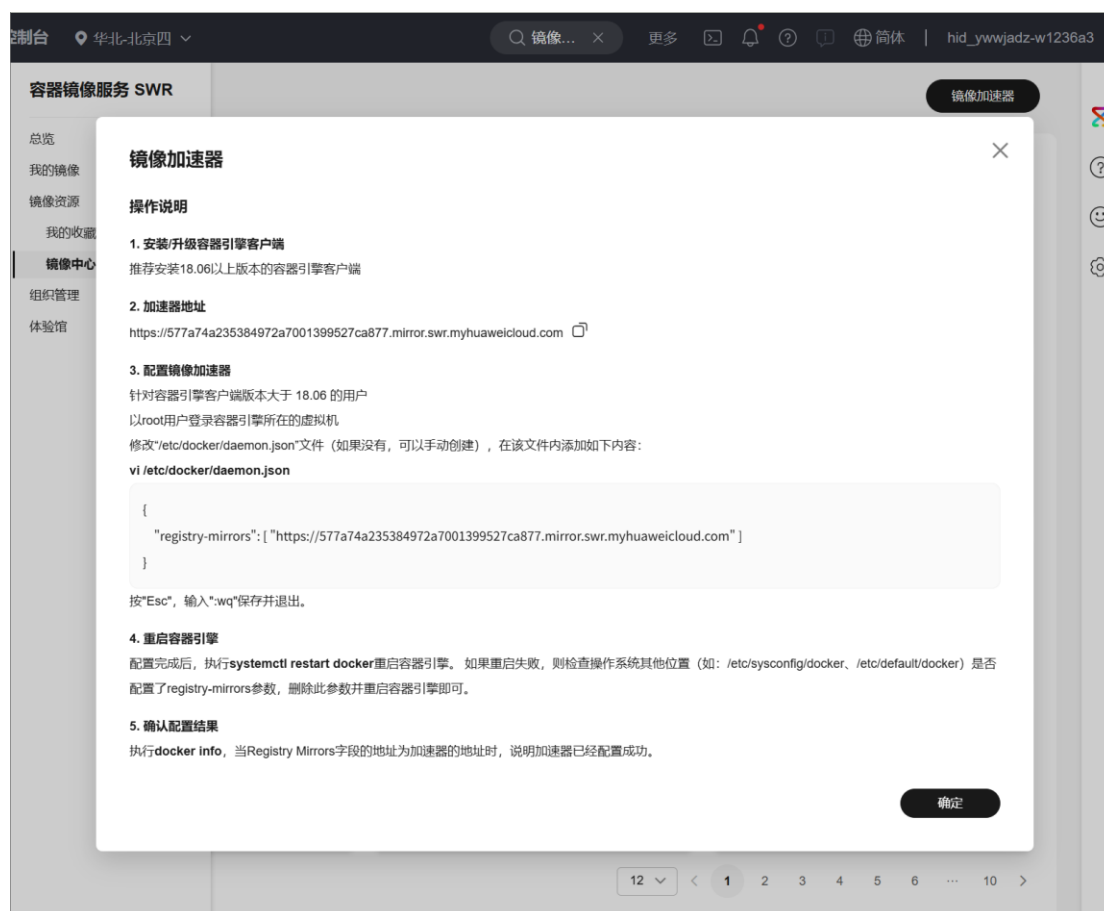


图 4.3 镜像加速器配置图

镜像上传前需要完成本地 Docker 环境的登录认证。如图 4.4 所示，在 SWR

控制台生成临时的 docker login 指令，复制该指令到 ECS 实例的 Docker 环境中执行。认证成功后 Docker 客户端即可获得推送和拉取镜像的权限，为后续的镜像上传做好准备。这一步骤需要特别注意认证信息的有效期，避免因认证过期导致操作失败。

```
[root@ecs-oa target]# docker images
REPOSITORY                                TAG      IMAGE ID      CREATED        SIZE
oa-kunpeng                               1.0      e29a592d6016  3 minutes ago  147MB
hello-world                              latest   1b44b5a3e06a  6 weeks ago    10.1kB
openjdk                                   8-alpine a3562aa0b991  6 years ago    105MB
swr.cn-north-4.myhuaweicloud.com/library/hello-world latest   4ab4c602aa5e  7 years ago    1.84kB
[root@ecs-oa target]# docker login -u cn-north-4@HST3WJ0JLW709VRUXT8D -p 41bf95a87a7e9d66ccb3b049c0c4cb5da121942c6
50c336b24565aed6c09be7b swr.cn-north-4.myhuaweicloud.com
WARNING! Using --password via the CLI is insecure. Use --password-stdin.
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[root@ecs-oa target]# docker tag oa-kunpeng:1.0 swr.cn-north-4.myhuaweicloud.com/innolegend/oa-kunpeng:1.0
[root@ecs-oa target]# docker push swr.cn-north-4.myhuaweicloud.com/innolegend/oa-kunpeng:1.0
The push refers to repository [swr.cn-north-4.myhuaweicloud.com/innolegend/oa-kunpeng]
88f652bde6d8: Pushed
ceaf9e1ebef5: Pushed
9b9b7f3d56a0: Pushed
f1b5933fe4b5: Pushed
1.0: digest: sha256:ad99ea03bf8995eac9b8f66363561e3da5c5643ebb8c5f346baa1942eb463c13 size: 1159
[root@ecs-oa target]#
```

图 4.4 镜像推送实操图

完成认证后需要对本地产建的 OA 系统镜像进行标签重命名。使用 docker tag 命令为镜像添加符合 SWR 规范的完整标签，包括镜像仓库地址、组织名称、镜像名称和版本号。标签格式必须严格按照 swr.cn-north-4.myhuaweicloud.com/{组织名称}/{镜像名称}:{版本号} 的规范设置，确保推送的目标地址准确无误。

如图 4.5 所示，镜像推送操作使用 docker push 命令将本地的 OA 系统镜像上传到 SWR 仓库。推送过程中会显示上传进度和层信息，通过分层传输机制提高上传效率。推送完成后在 SWR 控制台的镜像仓库页面可以查看已上传的镜像详情，包括镜像大小、更新时间、安全扫描状态等信息。

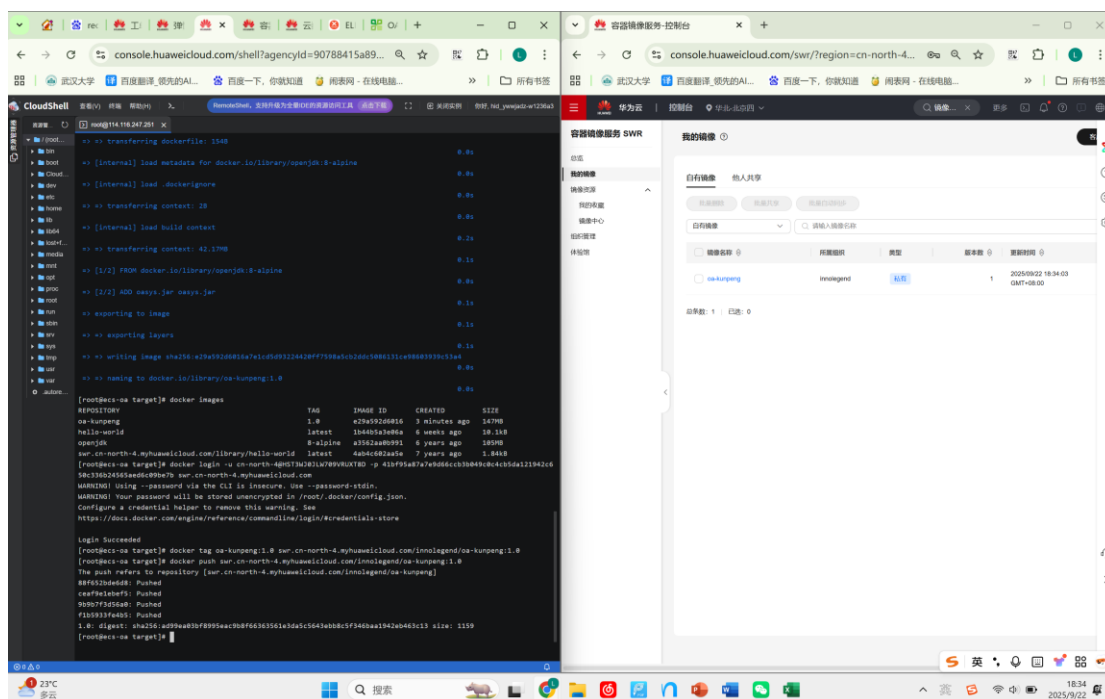


图 4.5 镜像推送验证图

4.3 CCE 集群部署

如图 4.6 所示，首先登录华为云控制台进入 CCE 服务页面，选择创建标准版集群。关键配置参数包括集群名称设置为 **oa-cluster**，版本选择 **v1.23** 及以上稳定版本，网络模型选择 **VPC** 网络确保与现有网络环境兼容。高可用选项启用多可用区部署，选择至少两个可用区以提高集群可靠性。

网络配置需要特别注意与现有 VPC 的匹配，选择 OA 系统所在的 VPC 和子网，配置合适的容器网段和服务网段，避免与现有网络产生冲突。安全组采用集群默认安全组并添加必要的端口规则，开放 80、8088 等应用端口和 30000-32767 的 NodePort 端口范围。



图 4.6 集群配置图

节点池配置采用弹性云服务器作为工作节点，选择通用计算优化型实例，规格配置为4vCPUs|8GB 内存以满足 OA 系统运行需求。操作系统选择 Ubuntu 20.04 或 CentOS 7.9，确保与容器运行时的兼容性。

数据盘配置为 100GB 高性能云硬盘，并启用自动扩容功能。节点数量初始设置为 2 个，启用弹性伸缩策略，根据 CPU 使用率自动调整节点数量，设置最小 1 个、最大 5 个节点的伸缩范围。为节点添加适当的标签，如 `app=oa-system`，便于后续的管理和调度。

如图 4.7 所示，在工作负载页面创建无状态部署，名称为 oa-system-deployment。容器配置选择从 SWR 镜像仓库拉取 oa-kunpeng 镜像，版本标签指定为最新稳定版。资源配额设置 CPU 请求 1 核、限制 2 核，内存请求 2GB、限制 4GB，确保应用运行稳定性。

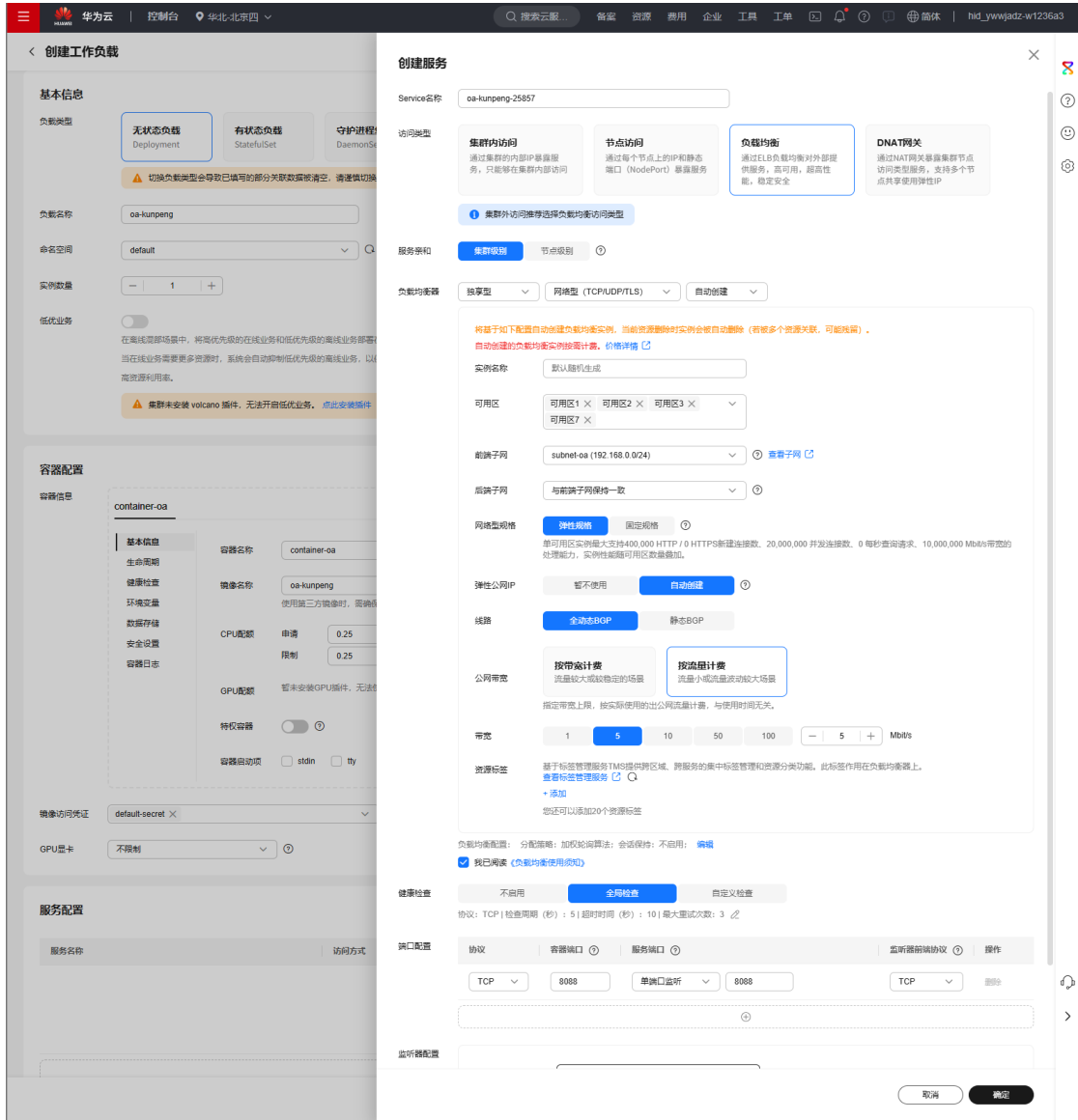


图 4.7 工作负载配置图

副本数初始设置为 2 个，并配置水平 Pod 自动伸缩，基于 CPU 使用率进行弹性伸缩，目标使用率设置为 70%。存储配置添加持久化卷声明，使用云硬盘存储类型，容量配置为 20GB，挂载到容器的日志目录。

服务配置选择负载均衡类型，创建 LoadBalancer 服务将 8088 端口暴露到公网。健康检查配置就绪探针和存活探针，设置适当的检查间隔和超时时间，确保应用的健康状态。

如图 4.8 所示，服务创建完成后，在服务页面查看自动分配的公网 IP 地址。通过浏览器访问 `http://<公网 IP>:8088`，验证 OA 系统登录页面能否正常打开。如图 4.9 所示，使用测试账户进行登录操作，验证各项功能是否正常运行。

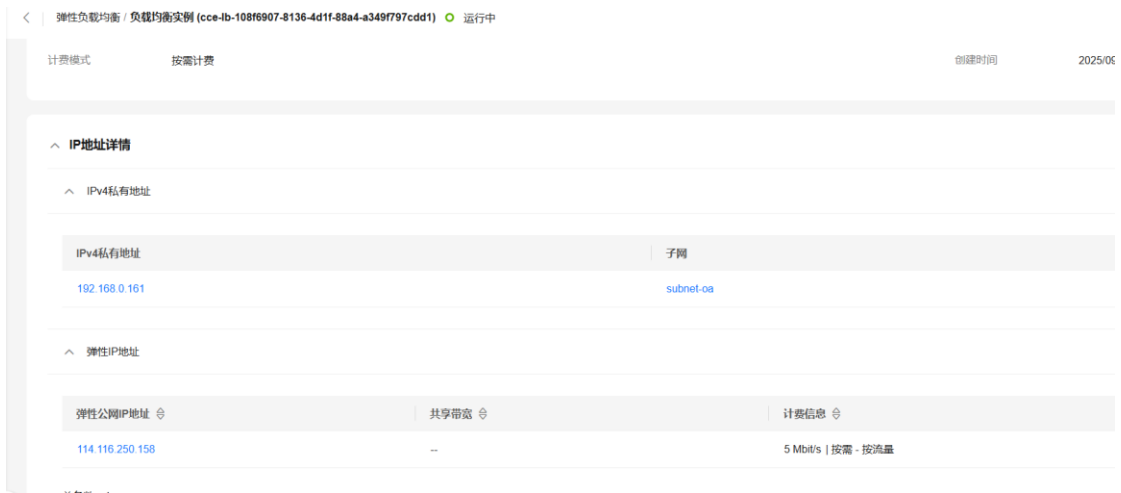


图 4.8 公网 IP 地址图

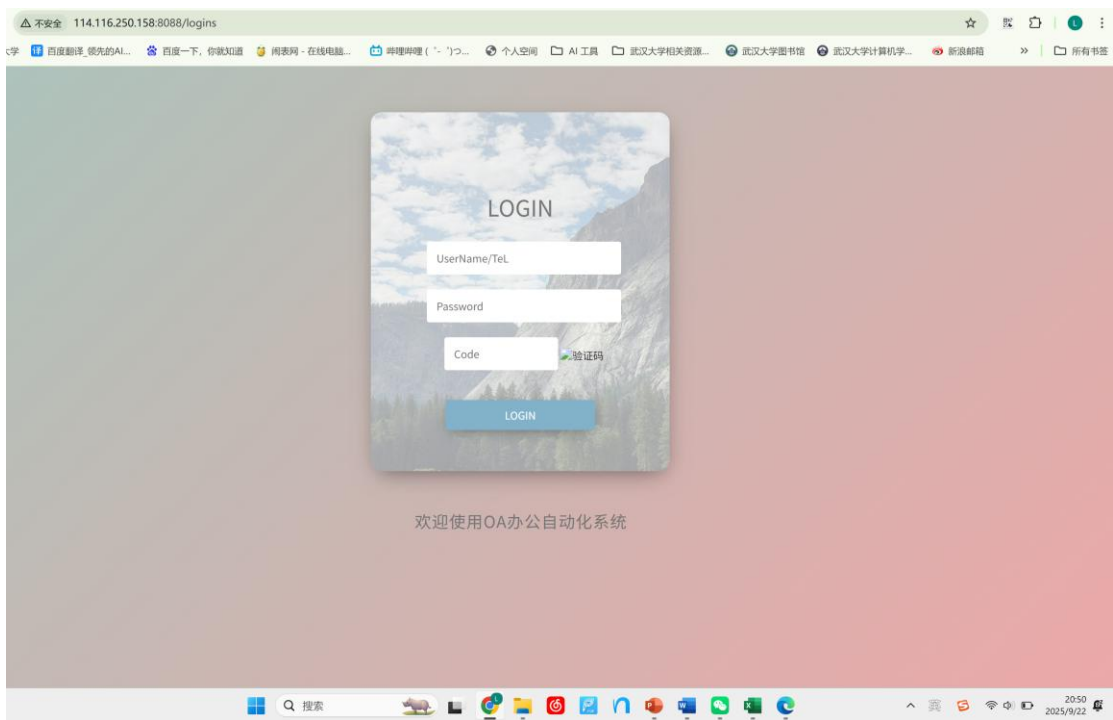


图 4.9 OA 系统登录验证图

检查工作负载的监控指标，包括 CPU 使用率、内存使用量和网络流量等关键指标。查看容器日志确认没有错误信息，验证应用是否正常启动并处理请求。配置域名解析将自有域名指向负载均衡 IP，启用 HTTPS 证书实现安全访问。通过 CCE 控制台的监控中心观察集群运行状态，包括节点资源使用情况、

Pod 运行状态和服务流量指标。设置告警规则对异常情况进行监控，确保 OA 系统的稳定运行。最终完成所有部署验证后，记录部署参数和配置详情，为后续的运维管理提供基础。最终本实验完成，我的具体花费如图 4.10 所示。

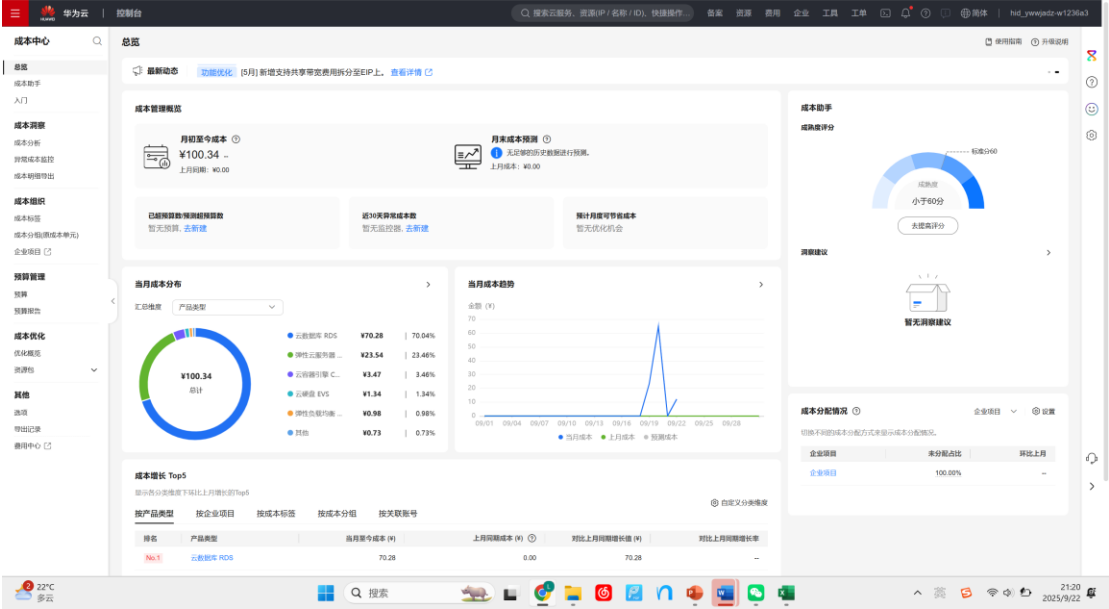


图 4.10 云容器实验花费图

5 实验感受与总结

5.1 实验手册注意事项

以下几条为我实验中发现实验手册上的不足之处：

1. 在原理上，手册没有说明 Docker 的端口映射为什么会出现一次为 80:80，一次为 81:80 的情况，这在实际操作中可能导致理解上的困惑。
2. 部署 OA 系统时，务必使用 x86 架构。如果选择了其他架构（如鲲鹏系统），则需要进行复杂的架构转换工作。因此，若计划在实验一的基础上进行本实验，需确保在实验一阶段就选用 x86 架构。
3. 目前通过 `yum install -y docker` 命令下载的 Docker 版本过低，不支持镜像加速功能，这会影响下载效率。建议采用第一部分提到的 `docker-ce` 社区版下载方式来获取最新及支持镜像加速的版本。
4. 实验中使用的镜像地址可以在官方网站上获取更新的版本，手册中提供的镜像地址可能已经不稳定或过期，建议及时查阅官网获取最新稳定的镜像。
5. 最大的问题是 DockerFile 里面的镜像版本。原手册中使用的 `java:openjdk-8-alpine` 镜像现在已不再支持；正确的版本应修改为 `openjdk:8-alpine`，否则会导致镜像构建失败。

5.2 实验感受

本次实验整体感受比较难受。其中一个主要原因是第二部分（可能是指容器化部署 OA 系统）相当于将实验一（应用部署）的内容从头到尾又做了一遍，造成了重复劳动。此外，在实验过程中，由于 Java 镜像仓库的问题，我一度误以为是网络故障，因此在问题排查上浪费了大约三个小时宝贵时间。这些意外状况使得实验过程的顺畅性大打折扣。

5.3 实验总结

本次《云容器实验》旨在深入理解容器技术，并亲手实践如何将传统应用（如 OA 系统）进行容器化部署及管理。通过本次实验，我不仅掌握了 Docker 的基本操作，包括镜像的构建、管理、容器的生命周期管理，还尝试了不同参数配置

和 Dockerfile 来部署容器化应用。主要收获如下：

深刻理解容器化部署流程：实践了从环境准备、Docker 环境搭建到应用镜像构建、容器运行的完整过程，对容器技术在云上应用的部署优势有了更直观的认识。

面对问题解决能力提升：实验中遇到的各种问题（如 Docker 版本、镜像源、Dockerfile 语法及镜像版本不兼容等）促使我主动查阅资料、分析日志，提升了独立解决问题和调试错误的能力。特别是关于 `java:openjdk-8-alpine` 与 `openjdk:8-alpine` 的镜像版本差异，是本次实验中一个重要的经验教训，提醒了在实际开发中需关注官方文档和镜像维护状态。